

Differential Evolution (DE)

Mutation, Crossover, Selection, and One-Generation Example

Evolutionary Algorithms

Outline

1. Introduction

1. Introduction

- ▶ Target vector, mutant vector, and trial vector

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

Outline

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

Outline

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

- ▶ Working principle through Rosenbrock function

Outline

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

- ▶ Working principle through Rosenbrock function
- ▶ Initial population and evaluation

Outline

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

- ▶ Working principle through Rosenbrock function
- ▶ Initial population and evaluation
- ▶ Generation of mutant vectors

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

- ▶ Working principle through Rosenbrock function
- ▶ Initial population and evaluation
- ▶ Generation of mutant vectors
- ▶ Generation of trial vectors

Outline

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

- ▶ Working principle through Rosenbrock function
- ▶ Initial population and evaluation
- ▶ Generation of mutant vectors
- ▶ Generation of trial vectors
- ▶ Greedy selection and new population

Outline

1. Introduction

- ▶ Target vector, mutant vector, and trial vector
- ▶ Mutation, crossover, and selection
- ▶ Variants of DE

2. DE Algorithm

- ▶ Working principle through Rosenbrock function
- ▶ Initial population and evaluation
- ▶ Generation of mutant vectors
- ▶ Generation of trial vectors
- ▶ Greedy selection and new population

3. Closure

- ▶ Differential Evolution (DE) is a population-based evolutionary optimization technique.

Introduction

- ▶ Differential Evolution (DE) is a population-based evolutionary optimization technique.
- ▶ It was proposed by R. Storn and K. Price for global optimization over continuous spaces.

Introduction

- ▶ Differential Evolution (DE) is a population-based evolutionary optimization technique.
- ▶ It was proposed by R. Storn and K. Price for global optimization over continuous spaces.
- ▶ DE is mainly used for nonlinear, non-differentiable, and continuous optimization problems.

Introduction

- ▶ Differential Evolution (DE) is a population-based evolutionary optimization technique.
- ▶ It was proposed by R. Storn and K. Price for global optimization over continuous spaces.
- ▶ DE is mainly used for nonlinear, non-differentiable, and continuous optimization problems.
- ▶ Similar to GA and PSO, DE works with a population of candidate solutions.

Introduction

- ▶ Differential Evolution (DE) is a population-based evolutionary optimization technique.
- ▶ It was proposed by R. Storn and K. Price for global optimization over continuous spaces.
- ▶ DE is mainly used for nonlinear, non-differentiable, and continuous optimization problems.
- ▶ Similar to GA and PSO, DE works with a population of candidate solutions.
- ▶ However, DE creates new solutions using **differences between existing vectors**.

Main Idea of DE

- ▶ Suppose we have a population of solution vectors.

Main Idea of DE

- ▶ Suppose we have a population of solution vectors.
- ▶ DE randomly chooses some vectors from the population.

Main Idea of DE

- ▶ Suppose we have a population of solution vectors.
- ▶ DE randomly chooses some vectors from the population.
- ▶ It subtracts two vectors to form a **difference vector**.

Main Idea of DE

- ▶ Suppose we have a population of solution vectors.
- ▶ DE randomly chooses some vectors from the population.
- ▶ It subtracts two vectors to form a **difference vector**.
- ▶ This difference vector gives a direction and step size for exploration.

Main Idea of DE

- ▶ Suppose we have a population of solution vectors.
- ▶ DE randomly chooses some vectors from the population.
- ▶ It subtracts two vectors to form a **difference vector**.
- ▶ This difference vector gives a direction and step size for exploration.

Simple idea

$$\text{new vector} = \text{base vector} + F \times \text{difference vector}$$

Main Idea of DE

- ▶ Suppose we have a population of solution vectors.
- ▶ DE randomly chooses some vectors from the population.
- ▶ It subtracts two vectors to form a **difference vector**.
- ▶ This difference vector gives a direction and step size for exploration.

Simple idea

$$\text{new vector} = \text{base vector} + F \times \text{difference vector}$$

Here, F controls how strongly the difference vector is used.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.
- ▶ Unlike PSO, DE does not use velocity and personal/global best movement.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.
- ▶ Unlike PSO, DE does not use velocity and personal/global best movement.
- ▶ DE perturbs one vector by adding the weighted difference of two randomly selected vectors.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.
- ▶ Unlike PSO, DE does not use velocity and personal/global best movement.
- ▶ DE perturbs one vector by adding the weighted difference of two randomly selected vectors.

Target vector

Mutant vector

Trial vector

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.
- ▶ Unlike PSO, DE does not use velocity and personal/global best movement.
- ▶ DE perturbs one vector by adding the weighted difference of two randomly selected vectors.

Target vector

Mutant vector

Trial vector

- ▶ **Target vector:** the current vector being improved.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.
- ▶ Unlike PSO, DE does not use velocity and personal/global best movement.
- ▶ DE perturbs one vector by adding the weighted difference of two randomly selected vectors.

Target vector

Mutant vector

Trial vector

- ▶ **Target vector:** the current vector being improved.
- ▶ **Mutant vector:** generated using a base vector and a difference vector.

Differential Evolution (DE)

- ▶ DE starts with a population of vectors or solutions.
- ▶ Unlike GA, DE does not mainly depend on crossover and mutation of a single chromosome.
- ▶ Unlike PSO, DE does not use velocity and personal/global best movement.
- ▶ DE perturbs one vector by adding the weighted difference of two randomly selected vectors.

Target vector

Mutant vector

Trial vector

- ▶ **Target vector:** the current vector being improved.
- ▶ **Mutant vector:** generated using a base vector and a difference vector.
- ▶ **Trial vector:** generated by mixing the target vector and mutant vector.

Nomenclature of Canonical DE

Parameter	Canonical DE	General notation
Population member / solution	i	i
Population size	NP	N
Generation counter	G	t
Maximum generations	G_{max}	T
Target vector	$\mathbf{x}_{i,G}$	$\mathbf{x}_i^{(t)}$
Mutant vector	$\mathbf{v}_{i,G+1}$	$\mathbf{v}_i^{(t+1)}$
Trial vector	$\mathbf{u}_{i,G+1}$	$\mathbf{u}_i^{(t+1)}$
Vector size	D	n
Crossover probability	CR	p_c

Nomenclature of Canonical DE

Parameter	Canonical DE	General notation
Population member / solution	i	i
Population size	NP	N
Generation counter	G	t
Maximum generations	G_{max}	T
Target vector	$\mathbf{x}_{i,G}$	$\mathbf{x}_i^{(t)}$
Mutant vector	$\mathbf{v}_{i,G+1}$	$\mathbf{v}_i^{(t+1)}$
Trial vector	$\mathbf{u}_{i,G+1}$	$\mathbf{u}_i^{(t+1)}$
Vector size	D	n
Crossover probability	CR	p_c

For example:

$$\mathbf{x}_i^{(t)} = \left(x_{i1}^{(t)}, x_{i2}^{(t)}, \dots, x_{in}^{(t)} \right)^T$$

DE Main Steps

1. Initialize a random population.

DE Main Steps

1. Initialize a random population.
2. Evaluate the fitness of each vector.

DE Main Steps

1. Initialize a random population.
2. Evaluate the fitness of each vector.
3. For each target vector, generate a mutant vector.

DE Main Steps

1. Initialize a random population.
2. Evaluate the fitness of each vector.
3. For each target vector, generate a mutant vector.
4. Generate a trial vector using crossover.

DE Main Steps

1. Initialize a random population.
2. Evaluate the fitness of each vector.
3. For each target vector, generate a mutant vector.
4. Generate a trial vector using crossover.
5. Evaluate the trial vector.

DE Main Steps

1. Initialize a random population.
2. Evaluate the fitness of each vector.
3. For each target vector, generate a mutant vector.
4. Generate a trial vector using crossover.
5. Evaluate the trial vector.
6. Use greedy selection to decide whether the trial vector replaces the target vector.

DE Main Steps

1. Initialize a random population.
2. Evaluate the fitness of each vector.
3. For each target vector, generate a mutant vector.
4. Generate a trial vector using crossover.
5. Evaluate the trial vector.
6. Use greedy selection to decide whether the trial vector replaces the target vector.
7. Repeat until the stopping condition is satisfied.

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

- ▶ For each target vector $\mathbf{x}_i^{(t)}$, a mutant vector $\mathbf{v}_i^{(t+1)}$ is generated.

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

- For each target vector $\mathbf{x}_i^{(t)}$, a mutant vector $\mathbf{v}_i^{(t+1)}$ is generated.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

- ▶ For each target vector $\mathbf{x}_i^{(t)}$, a mutant vector $\mathbf{v}_i^{(t+1)}$ is generated.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

- ▶ r_1 , r_2 , and r_3 are randomly selected indices.

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

- ▶ For each target vector $\mathbf{x}_i^{(t)}$, a mutant vector $\mathbf{v}_i^{(t+1)}$ is generated.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

- ▶ r_1 , r_2 , and r_3 are randomly selected indices.
- ▶ They must be different from each other and different from the target index i .

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

- ▶ For each target vector $\mathbf{x}_i^{(t)}$, a mutant vector $\mathbf{v}_i^{(t+1)}$ is generated.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

- ▶ r_1 , r_2 , and r_3 are randomly selected indices.
- ▶ They must be different from each other and different from the target index i .
- ▶ F is called the **mutation factor** or **scaling factor**.

Mutant Vector Using Mutation

Purpose

Create a new vector for exploration.

- ▶ For each target vector $\mathbf{x}_i^{(t)}$, a mutant vector $\mathbf{v}_i^{(t+1)}$ is generated.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

- ▶ r_1 , r_2 , and r_3 are randomly selected indices.
- ▶ They must be different from each other and different from the target index i .
- ▶ F is called the **mutation factor** or **scaling factor**.

$$r_1 \neq r_2 \neq r_3, \quad i \notin \{r_1, r_2, r_3\}$$

Role of the Scaling Factor F

- ▶ The difference vector is:

$$\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}$$

Role of the Scaling Factor F

- ▶ The difference vector is:

$$\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}$$

- ▶ The scaling factor F controls the size of the move.

Role of the Scaling Factor F

- ▶ The difference vector is:

$$\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}$$

- ▶ The scaling factor F controls the size of the move.

Effect of F

Role of the Scaling Factor F

- ▶ The difference vector is:

$$\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}$$

- ▶ The scaling factor F controls the size of the move.

Effect of F

- ▶ Small F : smaller change and more local search.

Role of the Scaling Factor F

- ▶ The difference vector is:

$$\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}$$

- ▶ The scaling factor F controls the size of the move.

Effect of F

- ▶ Small F : smaller change and more local search.
- ▶ Large F : larger change and more exploration.

Role of the Scaling Factor F

- ▶ The difference vector is:

$$\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)}$$

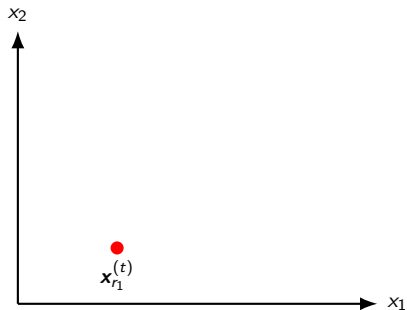
- ▶ The scaling factor F controls the size of the move.

Effect of F

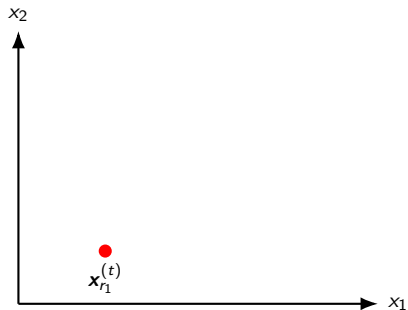
- ▶ Small F : smaller change and more local search.
- ▶ Large F : larger change and more exploration.
- ▶ Commonly, F is selected within a suitable range such as $0 \leq F \leq 2$.

Graphical Meaning of Mutation

Step 1: Difference vector

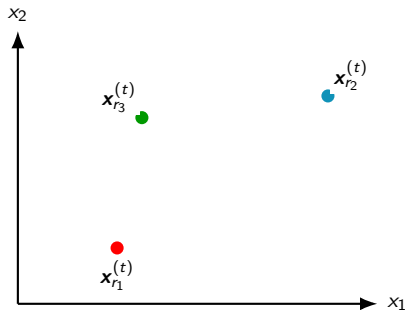


Step 2: Add scaled difference

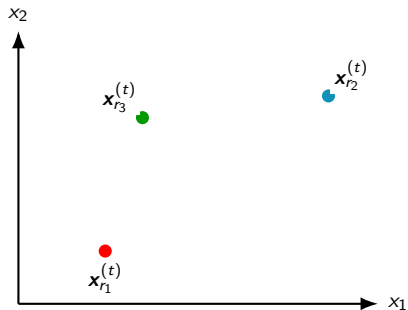


Graphical Meaning of Mutation

Step 1: Difference vector

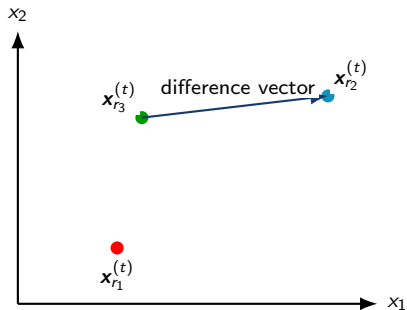


Step 2: Add scaled difference

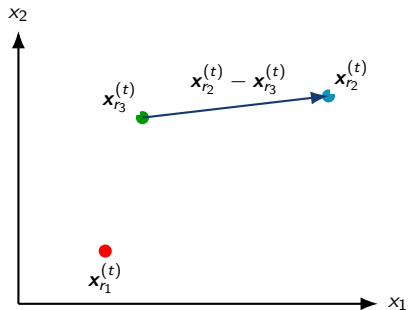


Graphical Meaning of Mutation

Step 1: Difference vector

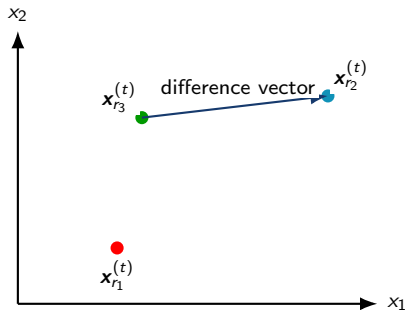


Step 2: Add scaled difference

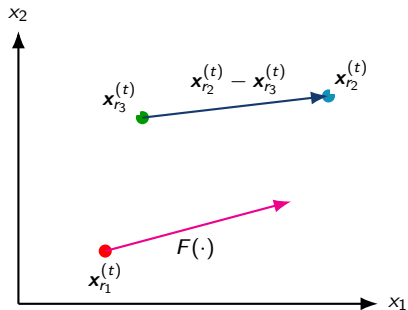


Graphical Meaning of Mutation

Step 1: Difference vector

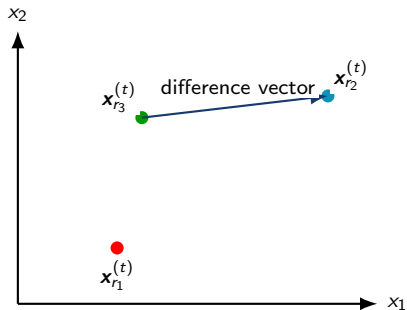


Step 2: Add scaled difference

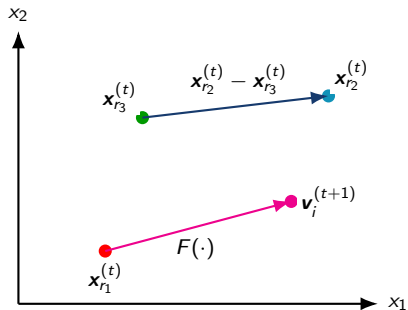


Graphical Meaning of Mutation

Step 1: Difference vector

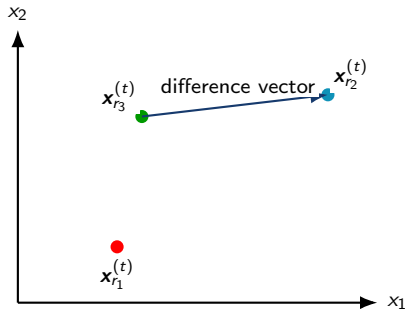


Step 2: Add scaled difference

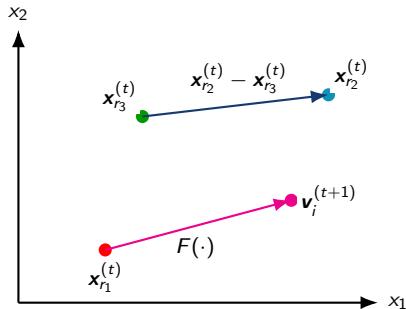


Graphical Meaning of Mutation

Step 1: Difference vector



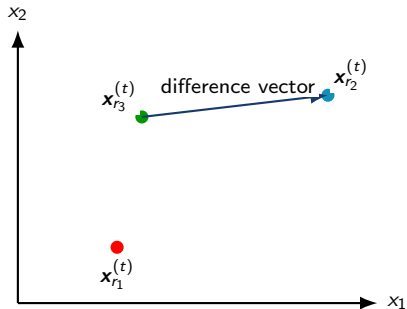
Step 2: Add scaled difference



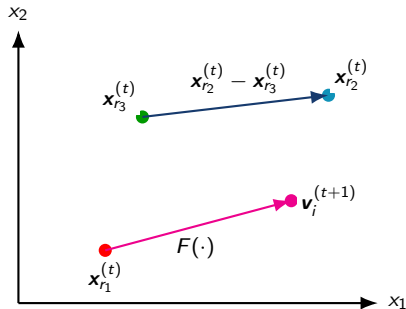
$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

Graphical Meaning of Mutation

Step 1: Difference vector



Step 2: Add scaled difference



$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{r_1}^{(t)} + F \left(\mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} \right)$$

The mutant vector is obtained by adding a scaled difference vector to a base vector.

Trial Vector Using Crossover

Purpose

Create a candidate vector by mixing the target vector and mutant vector.

Trial Vector Using Crossover

Purpose

Create a candidate vector by mixing the target vector and mutant vector.

- ▶ The trial vector is denoted by $\mathbf{u}_i^{(t+1)}$.

Trial Vector Using Crossover

Purpose

Create a candidate vector by mixing the target vector and mutant vector.

- ▶ The trial vector is denoted by $\mathbf{u}_i^{(t+1)}$.
- ▶ Each variable of the trial vector is copied either from the mutant vector or from the target vector.

Trial Vector Using Crossover

Purpose

Create a candidate vector by mixing the target vector and mutant vector.

- ▶ The trial vector is denoted by $\mathbf{u}_i^{(t+1)}$.
- ▶ Each variable of the trial vector is copied either from the mutant vector or from the target vector.

$$u_{ij}^{(t+1)} = \begin{cases} v_{ij}^{(t+1)}, & \text{if } rand_j \leq p_c \text{ or } j = j_{rand}, \\ x_{ij}^{(t)}, & \text{otherwise.} \end{cases}$$

Trial Vector Using Crossover

Purpose

Create a candidate vector by mixing the target vector and mutant vector.

- ▶ The trial vector is denoted by $\mathbf{u}_i^{(t+1)}$.
- ▶ Each variable of the trial vector is copied either from the mutant vector or from the target vector.

$$u_{ij}^{(t+1)} = \begin{cases} v_{ij}^{(t+1)}, & \text{if } rand_j \leq p_c \text{ or } j = j_{rand}, \\ x_{ij}^{(t)}, & \text{otherwise.} \end{cases}$$

- ▶ p_c is the crossover probability.

Trial Vector Using Crossover

Purpose

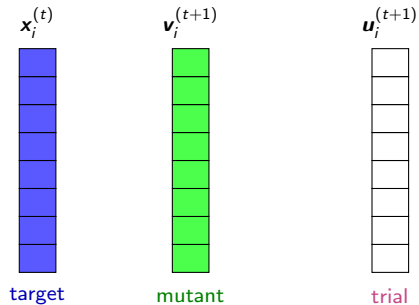
Create a candidate vector by mixing the target vector and mutant vector.

- ▶ The trial vector is denoted by $\mathbf{u}_i^{(t+1)}$.
- ▶ Each variable of the trial vector is copied either from the mutant vector or from the target vector.

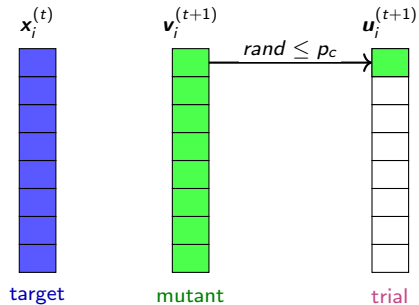
$$u_{ij}^{(t+1)} = \begin{cases} v_{ij}^{(t+1)}, & \text{if } rand_j \leq p_c \text{ or } j = j_{rand}, \\ x_{ij}^{(t)}, & \text{otherwise.} \end{cases}$$

- ▶ p_c is the crossover probability.
- ▶ j_{rand} ensures that at least one variable comes from the mutant vector.

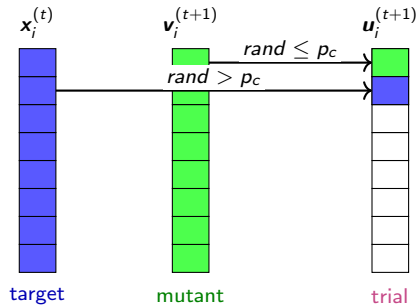
Graphical Illustration of Trial Vector



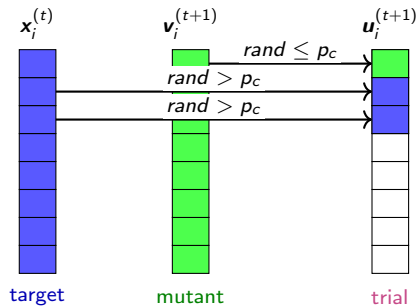
Graphical Illustration of Trial Vector



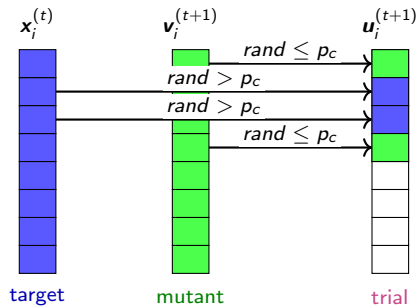
Graphical Illustration of Trial Vector



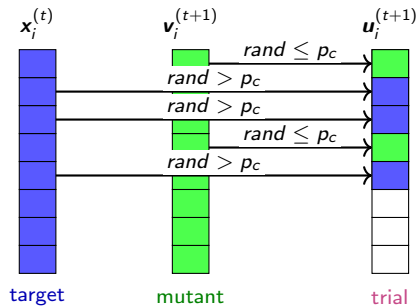
Graphical Illustration of Trial Vector



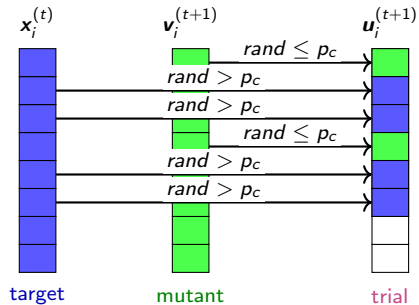
Graphical Illustration of Trial Vector



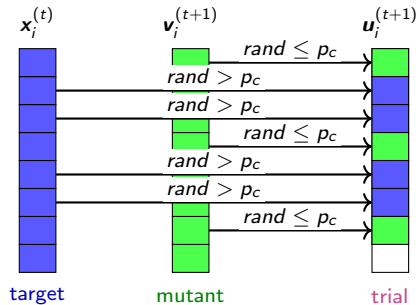
Graphical Illustration of Trial Vector



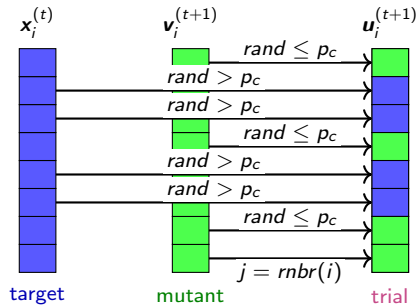
Graphical Illustration of Trial Vector



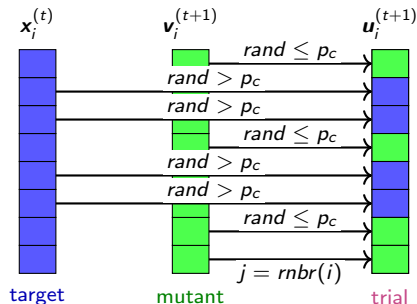
Graphical Illustration of Trial Vector



Graphical Illustration of Trial Vector

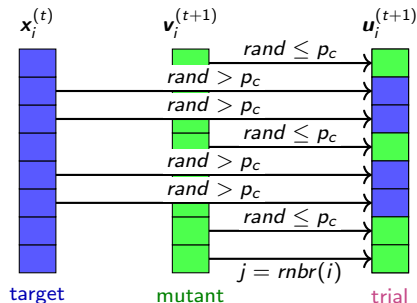


Graphical Illustration of Trial Vector



$$u_{ij}^{(t+1)} = \begin{cases} v_{ij}^{(t+1)}, & \text{if } rand \leq p_c \text{ or } j = rnbr(i), \\ x_{ij}^{(t)}, & \text{otherwise.} \end{cases}$$

Graphical Illustration of Trial Vector



$$u_{ij}^{(t+1)} = \begin{cases} v_{ij}^{(t+1)}, & \text{if } rand \leq p_c \text{ or } j = rnbr(i), \\ x_{ij}^{(t)}, & \text{otherwise.} \end{cases}$$

The trial vector is formed by mixing components from the target vector and the mutant vector.

Selection in DE

Purpose

Decide whether the trial vector should replace the target vector.

Purpose

Decide whether the trial vector should replace the target vector.

- ▶ The fitness of the trial vector is compared with the fitness of the target vector.

Purpose

Decide whether the trial vector should replace the target vector.

- ▶ The fitness of the trial vector is compared with the fitness of the target vector.
- ▶ For a minimization problem, the vector with smaller objective value is selected.

Purpose

Decide whether the trial vector should replace the target vector.

- ▶ The fitness of the trial vector is compared with the fitness of the target vector.
- ▶ For a minimization problem, the vector with smaller objective value is selected.

$$\mathbf{x}_i^{(t+1)} = \begin{cases} \mathbf{u}_i^{(t+1)}, & \text{if } f(\mathbf{u}_i^{(t+1)}) < f(\mathbf{x}_i^{(t)}), \\ \mathbf{x}_i^{(t)}, & \text{otherwise.} \end{cases}$$

Purpose

Decide whether the trial vector should replace the target vector.

- ▶ The fitness of the trial vector is compared with the fitness of the target vector.
- ▶ For a minimization problem, the vector with smaller objective value is selected.

$$\mathbf{x}_i^{(t+1)} = \begin{cases} \mathbf{u}_i^{(t+1)}, & \text{if } f(\mathbf{u}_i^{(t+1)}) < f(\mathbf{x}_i^{(t)}), \\ \mathbf{x}_i^{(t)}, & \text{otherwise.} \end{cases}$$

- ▶ This is called **greedy selection**.

Purpose

Decide whether the trial vector should replace the target vector.

- ▶ The fitness of the trial vector is compared with the fitness of the target vector.
- ▶ For a minimization problem, the vector with smaller objective value is selected.

$$\mathbf{x}_i^{(t+1)} = \begin{cases} \mathbf{u}_i^{(t+1)}, & \text{if } f(\mathbf{u}_i^{(t+1)}) < f(\mathbf{x}_i^{(t)}), \\ \mathbf{x}_i^{(t)}, & \text{otherwise.} \end{cases}$$

- ▶ This is called **greedy selection**.
- ▶ The new population is formed by selected vectors only.

Variants of DE

- ▶ DE variants are commonly written using the notation:

Variants of DE

- ▶ DE variants are commonly written using the notation:

DE/x/y/z

Variants of DE

- ▶ DE variants are commonly written using the notation:

DE/x/y/z

- ▶ x: vector used as the base vector, such as rand or best.

Variants of DE

- ▶ DE variants are commonly written using the notation:

$$\text{DE}/x/y/z$$

- ▶ x : vector used as the base vector, such as `rand` or `best`.
- ▶ y : number of difference vectors used.

Variants of DE

- ▶ DE variants are commonly written using the notation:

$$\text{DE}/x/y/z$$

- ▶ x : vector used as the base vector, such as `rand` or `best`.
- ▶ y : number of difference vectors used.
- ▶ z : crossover scheme, such as `bin` for binomial crossover.

Variants of DE

- ▶ DE variants are commonly written using the notation:

DE/x/y/z

- ▶ x: vector used as the base vector, such as `rand` or `best`.
- ▶ y: number of difference vectors used.
- ▶ z: crossover scheme, such as `bin` for binomial crossover.

Example

DE/rand/1/bin

uses a random base vector, one difference vector, and binomial crossover.

Example Variant: DE/best/2/bin

- ▶ In DE/best/2/bin, the best vector is used as the base vector.

Example Variant: DE/best/2/bin

- ▶ In DE/best/2/bin, the best vector is used as the base vector.
- ▶ Two difference vectors are used.

Example Variant: DE/best/2/bin

- ▶ In DE/best/2/bin, the best vector is used as the base vector.
- ▶ Two difference vectors are used.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{best}^{(t)} + F \left(\mathbf{x}_{r_1}^{(t)} + \mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} - \mathbf{x}_{r_4}^{(t)} \right)$$

Example Variant: DE/best/2/bin

- ▶ In DE/best/2/bin, the best vector is used as the base vector.
- ▶ Two difference vectors are used.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{best}^{(t)} + F \left(\mathbf{x}_{r_1}^{(t)} + \mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} - \mathbf{x}_{r_4}^{(t)} \right)$$

- ▶ This variant can exploit the current best solution more strongly.

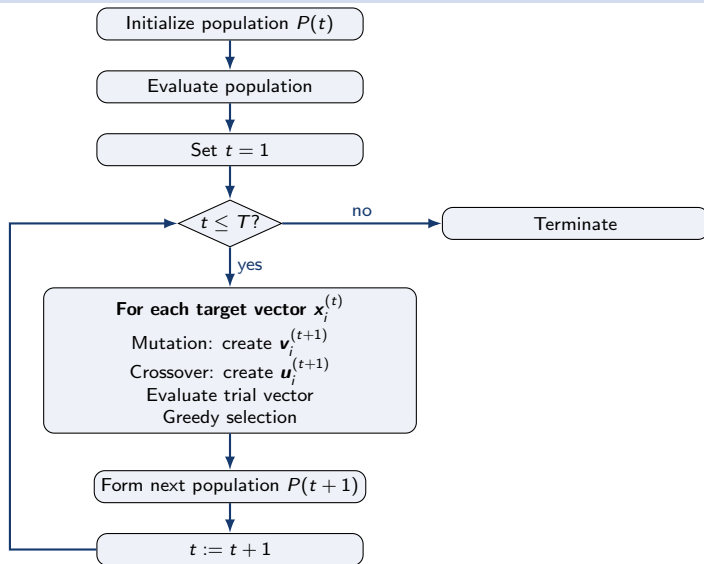
Example Variant: DE/best/2/bin

- ▶ In DE/best/2/bin, the best vector is used as the base vector.
- ▶ Two difference vectors are used.

$$\mathbf{v}_i^{(t+1)} = \mathbf{x}_{best}^{(t)} + F \left(\mathbf{x}_{r_1}^{(t)} + \mathbf{x}_{r_2}^{(t)} - \mathbf{x}_{r_3}^{(t)} - \mathbf{x}_{r_4}^{(t)} \right)$$

- ▶ This variant can exploit the current best solution more strongly.
- ▶ However, too much attraction toward the best solution may reduce diversity.

DE Flowchart



Explanation of DE Flowchart

- ▶ DE starts with a randomly generated population.
- ▶ Each solution in the population is called a target vector.
- ▶ For each target vector, DE generates a mutant vector.
- ▶ The mutant vector and target vector are mixed to form a trial vector.
- ▶ The trial vector is evaluated using the objective function.
- ▶ Greedy selection decides whether the trial vector replaces the target vector.
- ▶ These steps are repeated until the maximum number of generations is reached.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:
 - 5.1 For each target vector $i = 1, 2, \dots, N$:

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:
 - 5.1 For each target vector $i = 1, 2, \dots, N$:
 - 5.1.1 Generate mutant vector $\mathbf{v}_i^{(t+1)}$.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:
 - 5.1 For each target vector $i = 1, 2, \dots, N$:
 - 5.1.1 Generate mutant vector $\mathbf{v}_i^{(t+1)}$.
 - 5.1.2 Generate trial vector $\mathbf{u}_i^{(t+1)}$.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:
 - 5.1 For each target vector $i = 1, 2, \dots, N$:
 - 5.1.1 Generate mutant vector $\mathbf{v}_i^{(t+1)}$.
 - 5.1.2 Generate trial vector $\mathbf{u}_i^{(t+1)}$.
 - 5.1.3 Evaluate $\mathbf{u}_i^{(t+1)}$.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:
 - 5.1 For each target vector $i = 1, 2, \dots, N$:
 - 5.1.1 Generate mutant vector $\mathbf{v}_i^{(t+1)}$.
 - 5.1.2 Generate trial vector $\mathbf{u}_i^{(t+1)}$.
 - 5.1.3 Evaluate $\mathbf{u}_i^{(t+1)}$.
 - 5.1.4 Select between $\mathbf{x}_i^{(t)}$ and $\mathbf{u}_i^{(t+1)}$.

DE in Generalized EC Framework

Algorithm: Generalized Framework for DE

1. Choose solution representation.
2. Set $t = 1$ and maximum generations T .
3. Initialize random population $P(t)$.
4. Evaluate $P(t)$.
5. While $t \leq T$:
 - 5.1 For each target vector $i = 1, 2, \dots, N$:
 - 5.1.1 Generate mutant vector $\mathbf{v}_i^{(t+1)}$.
 - 5.1.2 Generate trial vector $\mathbf{u}_i^{(t+1)}$.
 - 5.1.3 Evaluate $\mathbf{u}_i^{(t+1)}$.
 - 5.1.4 Select between $\mathbf{x}_i^{(t)}$ and $\mathbf{u}_i^{(t+1)}$.
 - 5.2 Set $t := t + 1$.

Working Principle Through an Example

Rosenbrock Function

$$\text{Minimize } f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$-5 \leq x_1 \leq 5, \quad -5 \leq x_2 \leq 5$$

Working Principle Through an Example

Rosenbrock Function

$$\text{Minimize } f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$-5 \leq x_1 \leq 5, \quad -5 \leq x_2 \leq 5$$

- The optimum solution is:

$$\mathbf{x}^* = (1, 1)^T$$

Working Principle Through an Example

Rosenbrock Function

$$\text{Minimize } f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$-5 \leq x_1 \leq 5, \quad -5 \leq x_2 \leq 5$$

- ▶ The optimum solution is:

$$\mathbf{x}^* = (1, 1)^T$$

- ▶ The optimum function value is:

$$f(\mathbf{x}^*) = 0$$

Initial Population

Let the population size be $N = 8$ and set $t = 1$.

Index i	$\mathbf{x}_i^{(1)}$
1	$(2.212, 3.009)^T$
2	$(-2.289, -2.396)^T$
3	$(-2.393, -4.790)^T$
4	$(-0.639, 1.692)^T$
5	$(-3.168, 0.706)^T$
6	$(0.215, -2.350)^T$
7	$(-0.742, 1.934)^T$
8	$(-4.563, 4.791)^T$

Evaluate Population

- ▶ We evaluate the objective function for each vector.

Evaluate Population

► We evaluate the objective function for each vector.

► For solution 1:

$$\mathbf{x}_1^{(1)} = (2.212, 3.009)^T, \quad f(\mathbf{x}_1^{(1)}) = 357.154$$

Evaluate Population

- ▶ We evaluate the objective function for each vector.
- ▶ For solution 1:

$$\mathbf{x}_1^{(1)} = (2.212, 3.009)^T, \quad f(\mathbf{x}_1^{(1)}) = 357.154$$

Index i	$\mathbf{x}_i^{(1)}$	$f(\mathbf{x}_i^{(1)})$
1	$(2.212, 3.009)^T$	357.154
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, -4.790)^T$	11066.800
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 0.706)^T$	8718.166
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

Mutation Example: Random Indices

Assume:

$$F = 0.5$$

Mutation Example: Random Indices

Assume:

$$F = 0.5$$

For each target vector, choose r_1 , r_2 , and r_3 such that:

$$r_1 \neq r_2 \neq r_3, \quad i \notin \{r_1, r_2, r_3\}$$

Mutation Example: Random Indices

Assume:

$$F = 0.5$$

For each target vector, choose r_1 , r_2 , and r_3 such that:

$$r_1 \neq r_2 \neq r_3, \quad i \notin \{r_1, r_2, r_3\}$$

Target i	r_1	r_2	r_3
1	7	3	2
2	3	4	6
3	1	7	6
4	2	1	7
5	4	6	2
6	3	7	8
7	1	8	4
8	4	5	3

Mutant Vector: Target Vector 1

The mutation equation is:

$$\mathbf{v}_i^{(2)} = \mathbf{x}_{r_1}^{(1)} + F \left(\mathbf{x}_{r_2}^{(1)} - \mathbf{x}_{r_3}^{(1)} \right)$$

Mutant Vector: Target Vector 1

The mutation equation is:

$$\mathbf{v}_i^{(2)} = \mathbf{x}_{r_1}^{(1)} + F \left(\mathbf{x}_{r_2}^{(1)} - \mathbf{x}_{r_3}^{(1)} \right)$$

For target vector 1:

$$r_1 = 7, \quad r_2 = 3, \quad r_3 = 2$$

Mutant Vector: Target Vector 1

The mutation equation is:

$$\mathbf{v}_i^{(2)} = \mathbf{x}_{r_1}^{(1)} + F \left(\mathbf{x}_{r_2}^{(1)} - \mathbf{x}_{r_3}^{(1)} \right)$$

For target vector 1:

$$r_1 = 7, \quad r_2 = 3, \quad r_3 = 2$$

$$\mathbf{v}_1^{(2)} = \mathbf{x}_7^{(1)} + 0.5 \left(\mathbf{x}_3^{(1)} - \mathbf{x}_2^{(1)} \right)$$

Mutant Vector: Target Vector 1

The mutation equation is:

$$\mathbf{v}_i^{(2)} = \mathbf{x}_{r_1}^{(1)} + F \left(\mathbf{x}_{r_2}^{(1)} - \mathbf{x}_{r_3}^{(1)} \right)$$

For target vector 1:

$$r_1 = 7, \quad r_2 = 3, \quad r_3 = 2$$

$$\mathbf{v}_1^{(2)} = \mathbf{x}_7^{(1)} + 0.5 \left(\mathbf{x}_3^{(1)} - \mathbf{x}_2^{(1)} \right)$$

$$\mathbf{v}_1^{(2)} = \begin{pmatrix} -0.742 \\ 1.934 \end{pmatrix} + 0.5 \left[\begin{pmatrix} -2.393 \\ -4.790 \end{pmatrix} - \begin{pmatrix} -2.289 \\ -2.396 \end{pmatrix} \right]$$

Mutant Vector: Target Vector 1

The mutation equation is:

$$\mathbf{v}_i^{(2)} = \mathbf{x}_{r_1}^{(1)} + F \left(\mathbf{x}_{r_2}^{(1)} - \mathbf{x}_{r_3}^{(1)} \right)$$

For target vector 1:

$$r_1 = 7, \quad r_2 = 3, \quad r_3 = 2$$

$$\mathbf{v}_1^{(2)} = \mathbf{x}_7^{(1)} + 0.5 \left(\mathbf{x}_3^{(1)} - \mathbf{x}_2^{(1)} \right)$$

$$\mathbf{v}_1^{(2)} = \begin{pmatrix} -0.742 \\ 1.934 \end{pmatrix} + 0.5 \left[\begin{pmatrix} -2.393 \\ -4.790 \end{pmatrix} - \begin{pmatrix} -2.289 \\ -2.396 \end{pmatrix} \right]$$

$$\mathbf{v}_1^{(2)} = \begin{pmatrix} -0.793 \\ 0.737 \end{pmatrix}$$

Mutant Vector: Target Vector 2

For target vector 2:

$$r_1 = 3, \quad r_2 = 4, \quad r_3 = 6$$

Mutant Vector: Target Vector 2

For target vector 2:

$$r_1 = 3, \quad r_2 = 4, \quad r_3 = 6$$

$$\mathbf{v}_2^{(2)} = \mathbf{x}_3^{(1)} + 0.5 \left(\mathbf{x}_4^{(1)} - \mathbf{x}_6^{(1)} \right)$$

Mutant Vector: Target Vector 2

For target vector 2:

$$r_1 = 3, \quad r_2 = 4, \quad r_3 = 6$$

$$\mathbf{v}_2^{(2)} = \mathbf{x}_3^{(1)} + 0.5 \left(\mathbf{x}_4^{(1)} - \mathbf{x}_6^{(1)} \right)$$

$$\mathbf{v}_2^{(2)} = \begin{pmatrix} -2.393 \\ -4.790 \end{pmatrix} + 0.5 \left[\begin{pmatrix} -0.639 \\ 1.692 \end{pmatrix} - \begin{pmatrix} 0.215 \\ -2.350 \end{pmatrix} \right]$$

Mutant Vector: Target Vector 2

For target vector 2:

$$r_1 = 3, \quad r_2 = 4, \quad r_3 = 6$$

$$\mathbf{v}_2^{(2)} = \mathbf{x}_3^{(1)} + 0.5 \left(\mathbf{x}_4^{(1)} - \mathbf{x}_6^{(1)} \right)$$

$$\mathbf{v}_2^{(2)} = \begin{pmatrix} -2.393 \\ -4.790 \end{pmatrix} + 0.5 \left[\begin{pmatrix} -0.639 \\ 1.692 \end{pmatrix} - \begin{pmatrix} 0.215 \\ -2.350 \end{pmatrix} \right]$$

$$\mathbf{v}_2^{(2)} = \begin{pmatrix} -2.820 \\ -2.769 \end{pmatrix}$$

Mutant Vectors and Boundary Handling

- ▶ After mutation, each variable must remain within its bounds.

Mutant Vectors and Boundary Handling

- ▶ After mutation, each variable must remain within its bounds.
- ▶ If a variable is above the upper bound, set it to the upper bound.

Mutant Vectors and Boundary Handling

- ▶ After mutation, each variable must remain within its bounds.
- ▶ If a variable is above the upper bound, set it to the upper bound.
- ▶ If a variable is below the lower bound, set it to the lower bound.

Mutant Vectors and Boundary Handling

- ▶ After mutation, each variable must remain within its bounds.
- ▶ If a variable is above the upper bound, set it to the upper bound.
- ▶ If a variable is below the lower bound, set it to the lower bound.

Index i	Mutant vector before correction	Mutant vector after correction
1	$(-0.793, 0.737)^T$	$(-0.793, 0.737)^T$
2	$(-2.820, -2.769)^T$	$(-2.820, -2.769)^T$
3	$(1.734, 5.151)^T$	$(1.734, 5.000)^T$
4	$(-0.812, -1.858)^T$	$(-0.812, -1.858)^T$
5	$(0.613, 1.714)^T$	$(0.613, 1.714)^T$
6	$(-0.482, -6.218)^T$	$(-0.482, -5.000)^T$
7	$(0.250, 4.558)^T$	$(0.250, 4.558)^T$
8	$(-1.026, 4.439)^T$	$(-1.026, 4.439)^T$

Trial Vector: Crossover Setup

The trial vector is generated using:

$$u_{ij}^{(2)} = \begin{cases} v_{ij}^{(2)}, & \text{if } rand_j \leq p_c \text{ or } j = j_{rand}, \\ x_{ij}^{(1)}, & \text{otherwise.} \end{cases}$$

Trial Vector: Crossover Setup

The trial vector is generated using:

$$u_{ij}^{(2)} = \begin{cases} v_{ij}^{(2)}, & \text{if } rand_j \leq p_c \text{ or } j = j_{rand}, \\ x_{ij}^{(1)}, & \text{otherwise.} \end{cases}$$

Assume:

$$p_c = 0.5$$

Trial Vector: Crossover Setup

The trial vector is generated using:

$$u_{ij}^{(2)} = \begin{cases} v_{ij}^{(2)}, & \text{if } rand_j \leq p_c \text{ or } j = j_{rand}, \\ x_{ij}^{(1)}, & \text{otherwise.} \end{cases}$$

Assume:

$p_c = 0.5$		
Index i	$rand_1$	$rand_2$
1	0.459	0.122
2	0.268	0.684
3	0.792	0.149
4	0.674	0.265
5	0.695	0.282
6	0.691	0.708
7	0.293	0.493
8	0.755	0.873

Trial Vector: Target Vector 1

For target vector 1:

$$\mathbf{v}_1^{(2)} = (-0.793, 0.737)^T, \quad \mathbf{x}_1^{(1)} = (2.212, 3.009)^T$$

Trial Vector: Target Vector 1

For target vector 1:

$$\mathbf{v}_1^{(2)} = (-0.793, 0.737)^T, \quad \mathbf{x}_1^{(1)} = (2.212, 3.009)^T$$

For variable 1:

$$rand_1 = 0.459 \leq p_c = 0.5$$

Therefore:

$$u_{11}^{(2)} = v_{11}^{(2)} = -0.793$$

Trial Vector: Target Vector 1

For target vector 1:

$$\mathbf{v}_1^{(2)} = (-0.793, 0.737)^T, \quad \mathbf{x}_1^{(1)} = (2.212, 3.009)^T$$

For variable 1:

$$rand_1 = 0.459 \leq p_c = 0.5$$

Therefore:

$$u_{11}^{(2)} = v_{11}^{(2)} = -0.793$$

For variable 2:

$$rand_2 = 0.122 \leq p_c = 0.5$$

Therefore:

$$u_{12}^{(2)} = v_{12}^{(2)} = 0.737$$

Trial Vector: Target Vector 1

For target vector 1:

$$\mathbf{v}_1^{(2)} = (-0.793, 0.737)^T, \quad \mathbf{x}_1^{(1)} = (2.212, 3.009)^T$$

For variable 1:

$$rand_1 = 0.459 \leq p_c = 0.5$$

Therefore:

$$u_{11}^{(2)} = v_{11}^{(2)} = -0.793$$

For variable 2:

$$rand_2 = 0.122 \leq p_c = 0.5$$

Therefore:

$$u_{12}^{(2)} = v_{12}^{(2)} = 0.737$$

$$\mathbf{u}_1^{(2)} = (-0.793, 0.737)^T$$

Trial Vector: Target Vector 2

For target vector 2:

$$\mathbf{v}_2^{(2)} = (-2.820, -2.769)^T, \quad \mathbf{x}_2^{(1)} = (-2.289, -2.396)^T$$

Trial Vector: Target Vector 2

For target vector 2:

$$\mathbf{v}_2^{(2)} = (-2.820, -2.769)^T, \quad \mathbf{x}_2^{(1)} = (-2.289, -2.396)^T$$

For variable 1:

$$rand_1 = 0.268 \leq p_c = 0.5$$

Therefore:

$$u_{21}^{(2)} = v_{21}^{(2)} = -2.820$$

Trial Vector: Target Vector 2

For target vector 2:

$$\mathbf{v}_2^{(2)} = (-2.820, -2.769)^T, \quad \mathbf{x}_2^{(1)} = (-2.289, -2.396)^T$$

For variable 1:

$$rand_1 = 0.268 \leq p_c = 0.5$$

Therefore:

$$u_{21}^{(2)} = v_{21}^{(2)} = -2.820$$

For variable 2:

$$rand_2 = 0.684 > p_c = 0.5$$

Therefore:

$$u_{22}^{(2)} = x_{22}^{(1)} = -2.396$$

Trial Vector: Target Vector 2

For target vector 2:

$$\mathbf{v}_2^{(2)} = (-2.820, -2.769)^T, \quad \mathbf{x}_2^{(1)} = (-2.289, -2.396)^T$$

For variable 1:

$$rand_1 = 0.268 \leq p_c = 0.5$$

Therefore:

$$u_{21}^{(2)} = v_{21}^{(2)} = -2.820$$

For variable 2:

$$rand_2 = 0.684 > p_c = 0.5$$

Therefore:

$$u_{22}^{(2)} = x_{22}^{(1)} = -2.396$$

$$\mathbf{u}_2^{(2)} = (-2.820, -2.396)^T$$

Trial Vectors After Crossover

Index i	Trial vector $\mathbf{u}_i^{(2)}$	$f(\mathbf{u}_i^{(2)})$
1	$(-0.793, 0.737)^T$	4.379
2	$(-2.820, -2.396)^T$	10718.538
3	$(-2.393, 5.000)^T$	64.003
4	$(-0.639, -1.858)^T$	516.457
5	$(-3.168, 1.714)^T$	6937.656
6	$(0.215, -2.350)^T$	574.796
7	$(0.250, 4.558)^T$	2021.582
8	$(-4.563, 4.791)^T$	25731.235

Trial Vectors After Crossover

Index i	Trial vector $\mathbf{u}_i^{(2)}$	$f(\mathbf{u}_i^{(2)})$
1	$(-0.793, 0.737)^T$	4.379
2	$(-2.820, -2.396)^T$	10718.538
3	$(-2.393, 5.000)^T$	64.003
4	$(-0.639, -1.858)^T$	516.457
5	$(-3.168, 1.714)^T$	6937.656
6	$(0.215, -2.350)^T$	574.796
7	$(0.250, 4.558)^T$	2021.582
8	$(-4.563, 4.791)^T$	25731.235

These trial vectors are now compared with their corresponding target vectors.

Greedy Selection of DE

For minimization:

$$\mathbf{x}_i^{(2)} = \begin{cases} \mathbf{u}_i^{(2)}, & \text{if } f(\mathbf{u}_i^{(2)}) < f(\mathbf{x}_i^{(1)}), \\ \mathbf{x}_i^{(1)}, & \text{otherwise.} \end{cases}$$

Greedy Selection of DE

For minimization:

$$\mathbf{x}_i^{(2)} = \begin{cases} \mathbf{u}_i^{(2)}, & \text{if } f(\mathbf{u}_i^{(2)}) < f(\mathbf{x}_i^{(1)}), \\ \mathbf{x}_i^{(1)}, & \text{otherwise.} \end{cases}$$

Index i	$f(\mathbf{x}_i^{(1)})$	$f(\mathbf{u}_i^{(2)})$	Selected vector
1	357.154	4.379	trial
2	5843.569	10718.538	target
3	11066.800	64.003	trial
4	167.414	516.457	target
5	8718.166	6937.656	trial
6	574.796	574.796	target
7	194.618	2021.582	target
8	25731.235	25731.235	target

New Target Vectors for the Next Generation

Index i	$\mathbf{x}_i^{(2)}$	$f(\mathbf{x}_i^{(2)})$
1	$(-0.793, 0.737)^T$	4.379
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, 5.000)^T$	64.003
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 1.714)^T$	6937.656
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

New Target Vectors for the Next Generation

Index i	$\mathbf{x}_i^{(2)}$	$f(\mathbf{x}_i^{(2)})$
1	$(-0.793, 0.737)^T$	4.379
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, 5.000)^T$	64.003
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 1.714)^T$	6937.656
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

After one generation, some target vectors have been replaced by better trial vectors.

Compare Vectors After One Generation

Initial population		
i	$\mathbf{x}_i^{(1)}$	$f(\mathbf{x}_i^{(1)})$
1	$(2.212, 3.009)^T$	357.154
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, -4.790)^T$	11066.800
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 0.706)^T$	8718.166
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

New target vectors		
i	$\mathbf{x}_i^{(2)}$	$f(\mathbf{x}_i^{(2)})$
1	$(-0.793, 0.737)^T$	4.379
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, 5.000)^T$	64.003
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 1.714)^T$	6937.656
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

Compare Vectors After One Generation

Initial population		
i	$\mathbf{x}_i^{(1)}$	$f(\mathbf{x}_i^{(1)})$
1	$(2.212, 3.009)^T$	357.154
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, -4.790)^T$	11066.800
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 0.706)^T$	8718.166
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

New target vectors		
i	$\mathbf{x}_i^{(2)}$	$f(\mathbf{x}_i^{(2)})$
1	$(-0.793, 0.737)^T$	4.379
2	$(-2.289, -2.396)^T$	5843.569
3	$(-2.393, 5.000)^T$	64.003
4	$(-0.639, 1.692)^T$	167.414
5	$(-3.168, 1.714)^T$	6937.656
6	$(0.215, -2.350)^T$	574.796
7	$(-0.742, 1.934)^T$	194.618
8	$(-4.563, 4.791)^T$	25731.235

The best value after one generation is $f = 4.379$ at $\mathbf{x}_1^{(2)} = (-0.793, 0.737)^T$.

Key Observations from the Example

- ▶ DE creates a mutant vector using the weighted difference of two vectors.

Key Observations from the Example

- ▶ DE creates a mutant vector using the weighted difference of two vectors.
- ▶ Crossover creates a trial vector by mixing the mutant and target vectors.

Key Observations from the Example

- ▶ DE creates a mutant vector using the weighted difference of two vectors.
- ▶ Crossover creates a trial vector by mixing the mutant and target vectors.
- ▶ Greedy selection keeps the better vector for the next generation.

Key Observations from the Example

- ▶ DE creates a mutant vector using the weighted difference of two vectors.
- ▶ Crossover creates a trial vector by mixing the mutant and target vectors.
- ▶ Greedy selection keeps the better vector for the next generation.
- ▶ A vector can improve significantly even in one generation.

Key Observations from the Example

- ▶ DE creates a mutant vector using the weighted difference of two vectors.
- ▶ Crossover creates a trial vector by mixing the mutant and target vectors.
- ▶ Greedy selection keeps the better vector for the next generation.
- ▶ A vector can improve significantly even in one generation.
- ▶ Boundary handling is important because mutation can generate values outside the allowed range.

Important Control Parameters in DE

- **Population size N :** number of candidate solutions.

Important Control Parameters in DE

- ▶ **Population size N :** number of candidate solutions.
- ▶ **Scaling factor F :** controls the strength of mutation.

Important Control Parameters in DE

- ▶ **Population size N :** number of candidate solutions.
- ▶ **Scaling factor F :** controls the strength of mutation.
- ▶ **Crossover probability p_c :** controls how much information comes from the mutant vector.

Important Control Parameters in DE

- ▶ **Population size N :** number of candidate solutions.
- ▶ **Scaling factor F :** controls the strength of mutation.
- ▶ **Crossover probability p_c :** controls how much information comes from the mutant vector.
- ▶ **Maximum generations T :** stopping condition.

Important Control Parameters in DE

- ▶ **Population size N :** number of candidate solutions.
- ▶ **Scaling factor F :** controls the strength of mutation.
- ▶ **Crossover probability p_c :** controls how much information comes from the mutant vector.
- ▶ **Maximum generations T :** stopping condition.

General guideline

A good DE run needs a balance between exploration and exploitation.

- ▶ Differential Evolution is a simple and powerful population-based optimizer.

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation
 - ▶ crossover

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation
 - ▶ crossover
 - ▶ greedy selection

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation
 - ▶ crossover
 - ▶ greedy selection
- ▶ DE uses vector differences to guide the search.

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation
 - ▶ crossover
 - ▶ greedy selection
- ▶ DE uses vector differences to guide the search.
- ▶ The trial vector competes with the target vector.

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation
 - ▶ crossover
 - ▶ greedy selection
- ▶ DE uses vector differences to guide the search.
- ▶ The trial vector competes with the target vector.
- ▶ The better vector survives to the next generation.

- ▶ Differential Evolution is a simple and powerful population-based optimizer.
- ▶ It works mainly through three operations:
 - ▶ mutation
 - ▶ crossover
 - ▶ greedy selection
- ▶ DE uses vector differences to guide the search.
- ▶ The trial vector competes with the target vector.
- ▶ The better vector survives to the next generation.
- ▶ DE is especially useful for continuous optimization problems.